1. Aim

To implement a lexical analyzer for a subset of C in Flex that classifies lexemes into detailed token categories, prints the line number of each token, removes comments and detects lexical errors.

2. Program — prac2.l

```
%{
#include <stdio.h>
int errors = 0;
%}

%option noyywrap
%option yylineno

%%
"//".*                    { }
"/*"([^*]|\*+[^*/])*\*+"/"   { }

"if"|"else"|"switch"|"case"|"default"|"goto"                          { printf("%d\tCONTROL\t\t%s\n", yylineno,
yytext); }
"for"|"while"|"do"|"break"|"continue"                                { printf("%d\tLOOP\t\t%s\n", yylineno,
yytext); }
"int"|"float"|"char"|"double"|"void"|"long"|"short"|"signed"|"unsigned" { printf("%d\tDATATYPE\t%s\n",
yylineno, yytext); }
"auto"|"const"|"extern"|"register"|"static"|"volatile"                { printf("%d\tQUALIFIER\t%s\n",
yylineno, yytext); }
"struct"|"union"|"enum"|"typedef"|"sizeof"|"return"                   { printf("%d\tKEYWORD\t\t%s\n",
yylineno, yytext); }

[A-Za-z_][A-Za-z0-9_]*       { printf("%d\tIDENTIFIER\t%s\n", yylineno, yytext); }
[0-9]+\.[0-9]+               { printf("%d\tFLOAT_CONST\t%s\n", yylineno, yytext); }
[0-9]+                       { printf("%d\tINT_CONST\t%s\n", yylineno, yytext); }
\"[^\"\n]*\"                 { printf("%d\tSTRING\t\t%s\n", yylineno, yytext); }

"+"|"-"|"*"|"/"|"%"          { printf("%d\tARITH_OP\t%s\n", yylineno, yytext); }
"<="|">="|"=="|"!="|"<"|">" { printf("%d\tREL_OP\t\t%s\n", yylineno, yytext); }
"&&"|"||"|"!"                { printf("%d\tLOGICAL_OP\t%s\n", yylineno, yytext); }
"="                          { printf("%d\tASSIGN\t\t%s\n", yylineno, yytext); }
[(){}\[\];,]                 { printf("%d\tDELIMITER\t%s\n", yylineno, yytext); }

[ \t\n]+                     { }
.                            { printf("%d\tLEXICAL ERROR\tinvalid character '%s'\n", yylineno, yytext); errors+
+; }
%%

int main(int argc, char **argv) {
    yyin = fopen(argv[1], "r");
    printf("LINE\tTOKEN\t\tLEXEME\n");
    printf("----------------------------------------\n");
    yylex();
    printf("----------------------------------------\n");
    printf("total lexical errors: %d\n", errors);
    return 0;
}
```

3. Input File — input2.c

```
/* demo */
int a, b;
float sum = 0.0;
a = 5;
while (a < 10) {
    a = a + 1;
}
```

```
if (a >= 10 && b != 0) {
    sum = a - b;
}
b = a $ 2;
```

Line 1 has a comment to test comment removal and line 11 has the invalid character $ to test error detection.

4. Commands

```
flex prac2.l
gcc lex.yy.c -o prac2
./prac2 input2.c
```

5. Output

Fig. 1 — Token table, first part (lines 2 to 8)

```
gedit prac2.l
gedit input2.c
flex prac2.l
gcc lex.yy.c -o prac2
./prac2 input2.c
```

Fig. 2 — Second part, with the lexical error on line 11 and the error count

6. Observation

- The comment /* demo */ on line 1 produced no token, so output starts at line 2. Comment removal works.
- Line numbers in the output match the source, so yylineno is correct.
- On line 8, != is a single REL_OP and && a single LOGICAL_OP instead of two separate tokens each.
- = is reported as ASSIGN while == would be a REL_OP, so assignment and comparison are kept apart.
- The $ on line 11 matched no pattern and was reported as a lexical error. Scanning continued, and 2 and ; on the same line were still tokenised.
- Total lexical errors: 1, which matches the input.

7. Conclusion

The lexical analyzer classifies lexemes into thirteen categories, prints line numbers, removes comments and reports invalid characters without stopping. The token stream it produces is what the syntax analyzer takes as input in the next practical.